UAE REAL ESTATE PRICE PREDICTION

A MACHINE LEARNING APPROACH TO AUTOMATED PROPERTY APPRAISAL



- Supervised by: Asmath Famnaz

- Submission Date: 21st June 2026

THE PROBLEM

UAE Real Estate Is Hard to Price

01. **UAE is one of the world's most dynamic real estate markets**

02. **Prices vary enormously**

03. **Traditional valuation relies on a human appraiser**
- **Slow (1–3 days)**
- **Expensive (AED 1,500–5,000)**
- **May be biased**

04. **Demand for valuations is growing**



Solution
Machine Learning

- Traditional methods **can't scale**
- Comparable-sales methods are inaccurate
- Machine learning **discovers relationships** in large structured datasets **automatically**
- **Key advantages:**
  - Scalability
  - Non-linearity
  - Explainability
  - Consistency

Business Objectives:

- Build a **high-accuracy ML model** for UAE property price prediction
- **Deliver** a usable **prediction interface** for non-technical users
- **Provide** a documented, **reproducible**, and maintainable **methodology**

Dataset & Preprocessing

Dataset

- **5,058** residential-for-sale **listings** scraped from UAE real estate platforms
- **12** raw **columns**

Key preprocessing steps

- **Removed** constant/uninformative **columns**
- **Cleaned** non-numeric **values** ("7+" → 7 or "studio" → 0)
- **Extracted** listing **month** from timestamp as a seasonality **signal**
- **Imputed** missing **values** with median (inside a pipeline to prevent leakage)
- **Removed** top 10% **outliers** by price and size
- **One-hot encoded** furnishing: NO / PARTLY / YES
- **Log-transformed** the price target to reduce right-skew and stabilize training

```
title
displayAddress
bathrooms
bedrooms
addedOn
type
price
verified
priceDuration
sizeMin
furnishing
description
```

```python
df['bathroom_bedroom_ratio'] = df['bathrooms'] / df['bedrooms']

df['size_per_bedroom'] = df['sizeMin'] / df['bedrooms']

df['size_per_bathroom'] = df['sizeMin'] / df['bathrooms']

df['bedrooms_per_size'] = df['bedrooms'] / df['sizeMin']

df['bedrooms_bathrooms_interaction'] = df['bedrooms'] * df['bathrooms']
```

```python
# Preprocessing pipeline
preprocessor = ColumnTransformer([
    # Numerical features 1: impute -> scale
        ('num1', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ]), numeric_features1),

    # Numerical features 2: log -> scale
        ('num2', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('log', FunctionTransformer(np.log1p, validate=False)),
        ('scaler', StandardScaler())
    ]), numeric_features2),

    # Area, City -> Target Encoding
    ('city', Pipeline([
        ('extract', FunctionTransformer(extract_city)),
        ('encoder', TargetEncoder(target_type='continuous', random_state=42)),
    ]), ['displayAddress']),

    # Already encoded binary/categorical features
    ('passthrough', 'passthrough', passthrough_features)
])
```

Feature Engineering & Pipeline Architecture

5 Engineered Features

Pipeline Branches (ColumnTransformer)

- Numeric Group 1 (numerical features)

  Median imputation → StandardScaler

- Numeric Group 2 (size features)

  Median imputation → log1p → StandardScaler

- Location (area + city)

  TargetEncoder (with cross-fitting to prevent leakage)

- Binary flags (verified, furnishing dummies)

  pass-through

FOUR MODELS COMPARED

Models selected (simple → complex):

01. **Ridge Regression**
A „simple“ linear regression model

02. **Random Forest**
Ensemble of trees

03. **HistGradientBoosting**
Scikit-learn's LightGBM-inspired boosting

04. **CatBoost**
Most modern and state-of-the-art model

Evaluation strategy:

5-fold cross-validation | **Metric: R^2** on **log-scale** prices | **80/20** stratified **train/test split** (stratified by price)

Hyperparameter Tuning

```python
hgb_param_dist = {
    'model__learning_rate': np.logspace(-2, 0, 10),        # 0.01 to 1.0
    'model__max_iter': [100, 200, 300, 400, 500],
    'model__max_depth': [3, 5, 7, 9, 11, None],
    'model__l2_regularization': [0.0, 0.01, 0.1, 0.5, 1.0, 2.0],
    'model__min_samples_leaf': [1, 5, 10, 20],
    'model__max_leaf_nodes': [15, 31, 63, 127],
    'preprocessor__num1__imputer__strategy': ['mean', 'median'],
    'preprocessor__num2__imputer__strategy': ['mean', 'median'],
    'preprocessor__city__encoder__smooth': np.logspace(-2, 1, 10)
}

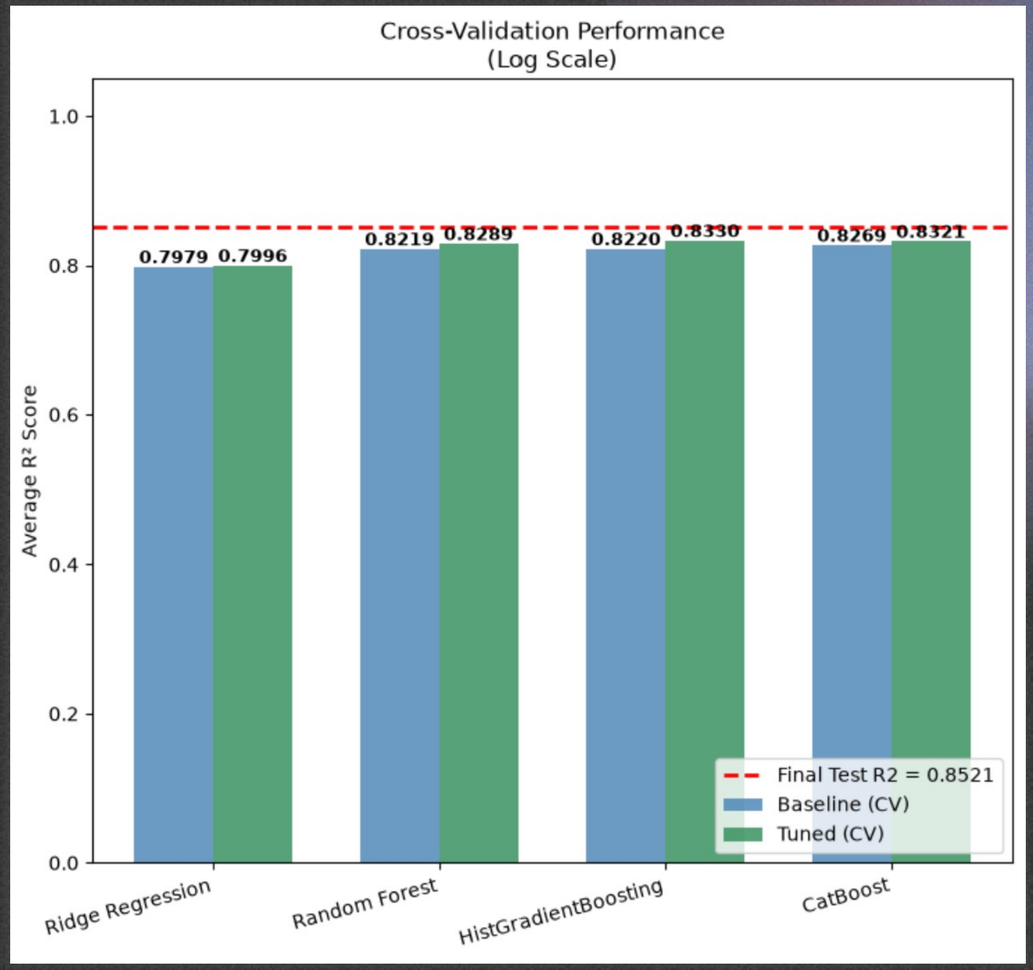
hgb_pipeline = models["HistGradientBoosting"]

hgb_random = RandomizedSearchCV(
    hgb_pipeline,
    param_distributions=hgb_param_dist,
    n_iter=100,
    cv=kf,
    scoring='r2',
    n_jobs=-1,
    verbose=3,
    random_state=42
)
```

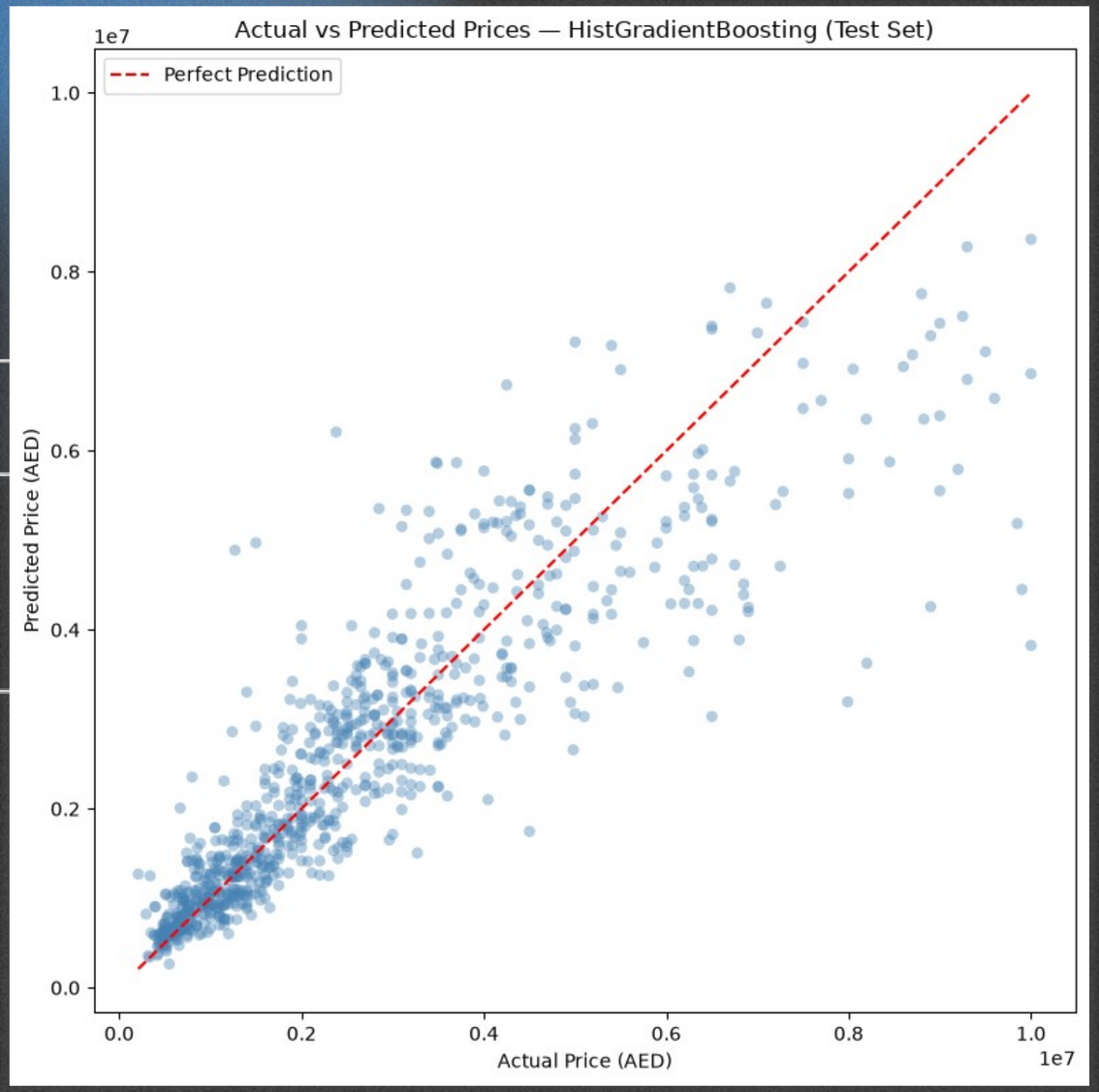
Method: **RandomizedSearchCV**

Results & Model Performance

Model	Baseline R2 (CV) (Log Scale)	Tuned R2 (CV) (Log Scale)
Ridge Regression	0.7979	0.7996
Random Forest	0.8219	0.8289
HistGradientBoosting	0.8220	0.8330
CatBoost	0.8269	0.8321



Test Set Performance




```
Final Model Performance Report (Tuned):
  - Selected Model           : HistGradientBoosting
  - Test R2  (Original AED) : 0.7836
  - Test RMSE (Original AED): 925,388 AED
  - Test MAE  (Original AED): 579,614 AED

  (Log-scale R2 for reference: 0.8521)
```

- **HGB retrained on full training set**
- **Evaluated once on held-out 20%:**

Business Impact

Four categories of value:

01. Time savings

02. Scale

03. Pricing guidance

Agents can use MAE as a confidence band around a listing price (e.g., ±12.5% on AED 2M properties).

04. Bias reduction

The model applies identical feature weightings to every property, eliminating individual agent subjectivity.

Limitations & Challenges

01. **Data limitations:**

- 5,058 records is **too small** for a market as diverse as the UAE
- Dataset contains **listing** prices, **not confirmed sale transaction** prices
- **Missing features:** floor level, view type, building age, proximity to landmarks

02. **Technical limitations:**

- Property descriptions
- **No feature importance** output from **HGB**
- Static model

03. **Ethical considerations:**

- Geographic bias

Lessons Learned

01.

Preprocessing is **everything**

02.

Log-transforming the target variable is **essential** for real estate

03.

Pipeline encapsulation **prevents** data leakage

04.

Tune preprocessing alongside model **hyperparameters**

05.

Stratified splitting by price is **essential**

Recommendations for Deployment

01.

Proposed architecture:

- **Store predictor.pkl** in a versioned **object store** (AWS S3 / Azure Blob) with access **logging**
- Expose a **REST API endpoint** (/predict
- **Integrate as an** "Estimated Value" **widget** on listing pages
- Run **A/B testing** on initial deployment and show model-predicted prices alongside manual prices

02.

Retraining plan:

- **Monitor live prediction** vs. eventual sale price to detect model drift early
- **Automate data ingestion** from UAE listing platforms



Future Work & Broader Applications

Proposed enhancements:

- **Transaction** price data
- **NLP** on descriptions
- **Geographic distance** features
- **SHAP** explainability
- **Automatic retraining** pipeline
- **GridSearchCV around** found **RandomSearchCV** parameters

Broader applications:

- **GCC** expansion
- Commercial real estate
- Rental price prediction

Summary
Q&A

What we built:

- A **supervised ML** model that explains **78%** of price **variance** on unseen data
- **Interactive implementation example**

Key takeaways:

- **Preprocessing**
- Meaningful **business value**
- **Limitations**

Thank you. Questions?